



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Detección y análisis de
ataques en sistemas
embebidos
Documentación Técnica**



Presentado por Sara Abejón Pérez
en Universidad de Burgos — 9 de junio
de 2026

Tutor: Jaime Andrés Rincón Arango y Daniel
Urda Muñoz

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	4
Apéndice B Especificación de Requisitos	11
B.1. Introducción	11
B.2. Objetivos generales	12
B.3. Catálogo de requisitos	12
B.4. Especificación de requisitos	15
Apéndice C Especificación de diseño	37
C.1. Introducción	37
C.2. Diseño de datos	37
C.3. Diseño arquitectónico	42
C.4. Diseño procedimental	45
Apéndice D Documentación técnica de programación	49
D.1. Introducción	49
D.2. Estructura de directorios	49
D.3. Manual del programador	51

D.4. Compilación, instalación y ejecución del proyecto	51
D.5. Pruebas del sistema	52
Apéndice E Documentación de usuario	55
E.1. Introducción	55
E.2. Requisitos de usuarios	55
E.3. Instalación	55
E.4. Manual del usuario	55
Apéndice F Anexo de sostenibilización curricular	57
F.1. Introducción	57
F.2. Competencias de sostenibilidad aplicadas	58
F.3. Conclusiones	59
Bibliografía	61

Índice de figuras

B.1. Diagrama de casos de uso	16
C.1. Diagrama Modelo Entidad-Relación	38
C.2. Diagrama relacional	40
C.3. Arquitectura MVC del Software.	43
C.4. Arquitectura por contenedores.	44
C.5. Diagrama de secuencia de la detección.	46

Índice de tablas

A.1. Coste de personal	5
A.2. Coste de hardware	5
A.3. Costes adicionales	6
A.4. Costes totales del proyecto	6
A.5. Licencias de librerías	9
A.6. Licencias de herramientas	9
A.7. Licencias de recursos	9
B.1. CU-1 Registro de usuarios.	17
B.2. CU-2 Inicio de sesión.	18
B.3. CU-3 Acceso de invitado.	19
B.4. CU-4 Gestión de cuenta.	19
B.5. CU-5 Modificar nombre de usuario.	20
B.6. CU-6 Modificar contraseña.	21
B.7. CU-7 Eliminar cuenta.	22
B.8. CU-8 Cerrar sesión.	23
B.9. CU-9 Gestionar dispositivos.	24
B.10.CU-10 Añadir dispositivo.	24
B.11.CU-11 Eliminar dispositivo.	25
B.12.CU-12 Gestionar modelos.	25
B.13.CU-13 Añadir modelo.	26
B.14.CU-14 Eliminar modelo.	27
B.15.CU-15 Limpieza de artefactos.	28
B.16.CU-16 Evaluar modelos.	29
B.17.CU-17 Laboratorio de detección.	30
B.18.CU-18 Monitorizar detección.	31
B.19.CU-19 Gestión de usuarios.	32
B.20.CU-20 Cambiar rol.	32

B.21.CU-21 Mostrar información de usuario.	33
B.22.CU-22 Eliminar artefacto.	34
B.23.CU-23 Eliminar usuario.	35
C.1. Diccionario de datos. Tabla correspondiente a la clase User . . .	40
C.2. Diccionario de datos. Tabla correspondiente a la clase Modelo . .	41
C.3. Diccionario de datos. Tabla correspondiente a la clase Dispositivo	41
C.4. Diccionario de datos. Tabla correspondiente a la clase File . . .	42

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Este documento tiene como objetivo detallar el plan de desarrollo seguido durante la ejecución de este proyecto. Proporciona una visión general de la organización, la metodología de trabajo aplicada, la estructuración del tiempo y el estudio de viabilidad económica y legal.

Para la gestión del proyecto se ha adoptado la metodología ágil **Scrum**. Este marco de trabajo resulta idóneo para entornos de desarrollo iterativos e incrementales, permitiendo una adaptación flexible a las necesidades del proyecto.

El ciclo de vida del proyecto se ha estructurado en sprints de una duración aproximada de dos semanas. Al finalizar cada iteración, se ha procurado alcanzar un incremento funcional del proyecto.

Toda la planificación, asignación de tareas y seguimiento del progreso se ha centralizado a través de la herramienta Git Projects, la cual ha servido como tablero Kanban para visualizar el estado de cada tarea.

A.2. Planificación temporal

Las tareas de desarrollo del proyecto se han dividido temporalmente de la siguiente manera:

Sprint 1 / Fase previa (18/12/2025 - 02/02/2026)

Se dedicó íntegramente a la realización de pruebas previas con diversos dispositivos microcontroladores y tecnologías alternativas. El objetivo principal fue evaluar el potencial de las distintas opciones para definir el contexto del proyecto. Durante esta fase se seleccionó el tipo de modelo de Inteligencia Artificial inicial a implementar y se estableció el hardware definitivo sobre el que se realizaría la computación on-edge: un M5Stack LLM 630 Compute Kit (AX630C).

Sprint 2 (04/02/2026 - 15/02/2026)

- Carga de datos de entrenamiento y validación.
- Creación del primer modelo de inteligencia artificial.

Sprint 3 (16/02/2026 - 01/03/2026)

- Creación y documentación del script para la detección en tiempo real (on-edge).
- Primera integración del modelo en el microcontrolador.

Sprint 4 (02/03/2026 - 15/03/2026)

- Planificación sobre la posible arquitectura de la aplicación web.
- Creación del entorno Docker para el proyecto.

Sprint 5 (16/03/2026 - 29/03/2026)

En este sprint se decidió cambiar el tipo de modelo a implementar, de un MLP al algoritmo Random Forest.

- Creación y documentación del código de entrenamiento, validación del modelo.
- Actualización del script de detección on-edge en el dispositivo.

Sprint 6 (30/03/2026 - 12/04/2026)

En este periodo se realizaron tareas dedicadas al comienzo del desarrollo y programación de la aplicación web. Se creó una plantilla base sobre la que comenzar el diseño de la web, y se implementó funcionalidad inicial y de prueba sobre la que construir el proyecto.

Sprint 7 (13/04/2026 - 26/04/2026)

Durante las dos semanas de duración de este sprint, hubo una dedicación exclusiva a la redacción y estructuración de la memoria del proyecto.

Sprint 8 (27/04/2026 - 10/05/2026)

El objetivo principal fue la inclusión de nuevas funcionalidades en la plataforma web. Entre ellas:

- Registro de usuarios.
- Inicio de sesión.
- Funcionalidades relacionadas con la pantalla de *Modelos*.

Sprint 9 (11/05/2026 - 24/05/2026)

Se implementaron mejoras en el diseño de la interfaz de la aplicación web y nuevas funcionalidades:

- Evaluación de modelos.
- Gestión de dispositivos.
- Laboratorio de detección.
- Funcionalidad de la cuenta de usuario.

Sprint 10 (25/05/2026 - 10/06/2026)

Los objetivos principales del sprint final son:

- Creación de nuevos modelos.
- Finalización de la web.

- Redacción del informe final.

Se crearon nuevos modelos RandomForest siguiendo diferentes estrategias de evaluación, ya que el modelo implementado solo se ajustaba a un tipo de tráfico específico, haciendo que la generalización no fuese posible.

A.3. Estudio de viabilidad

La viabilidad de un proyecto de software se refiere a su capacidad para ser completado con éxito y cumplir con los objetivos establecidos.

Viabilidad económica

Para determinar si el proyecto es viable en términos económicos, se analizan los costes asociados a su desarrollo y los beneficios en caso de comercialización.

Costes

- Costes de personal

En este apartado se abarcan los costes relacionados con la contratación de personal para el desarrollo del proyecto. Se ha estimado un total de 500 horas de trabajo invertidas en el proyecto. Tomando como referencia el salario bruto anual de un ingeniero junior en 2026 estipulado en aproximadamente 21.938€¹, lo que equivale a aproximadamente 10,98€/hora, aplicando las cotizaciones dentro del Régimen General de la Seguridad Social[1], el coste total sería:

¹Salario programador junior: <https://es.indeed.com/career/programador-junior/salaries>

Concepto		Coste €
Salario bruto por hora		10,98
Contingencias comunes	23,60 %	2,59
Desempleo	5,50 %	0,60
Fondo de garantía salarial	0,20 %	0,02
Formación	0,60 %	0,06
Mecanismo de Equidad Intergeneracional	0,75 %	0,08
Coste por hora		14,33
Total 500 horas		7.165 €

Tabla A.1: Coste de personal

■ Costes de hardware

En este apartado se abarcan los costes relacionados con todo el hardware que se ha necesitado para el desarrollo del proyecto. Este consta de un equipo portátil para el desarrollo y el microcontrolador para la fase experimental. El dispositivo M5Stack está valorado en 92€ aproximadamente (99,90\$), pero se considera completamente amortizado porque su adquisición fue hace más tiempo que la estimación de su vida útil. Por otro lado, el ordenador usado para el desarrollo del proyecto es un portátil Lenovo con un valor estimado de compra de 700€. Suponiendo una amortización lineal a 4 años, el coste correspondiente a los 6 meses de duración del proyecto es de 87,50€.

Concepto	Coste €	Coste amortizado €
M5Stack LLM630 Compute Kit	92	0
Lenovo IdeaPad Flex 5 14ABR8	700	87,50
Total		87,50 €

Tabla A.2: Coste de hardware

■ Costes de software

Todo el software utilizado en este proyecto es de código abierto y gratuito para uso personal, por lo que no hay costes asociados a licencias de software.

■ Otros costes

Se contemplan los gastos derivados del consumo eléctrico y de la conexión a Internet durante el desarrollo del proyecto.

Concepto	Coste €/mes
Consumo eléctrico	13,46
Internet	16,24
Coste por mes	29,70
Total	178,20 €

Tabla A.3: Costes adicionales

Total de costes

Teniendo en cuenta todos los costes asociados calculados, el coste total del proyecto asciende a:

Concepto	Coste €
Personal	7.165
Hardware	87,50
Software	0
Adicionales	178,20
Total	7.430,70 €

Tabla A.4: Costes totales del proyecto

Beneficios

Es necesario establecer alternativas para poder monetizar el proyecto, ya que el software creado se distribuye de forma gratuita y sin publicidad. Una posibilidad es el *modelo freemium*, que consiste en establecer una versión básica gratuita, que ofrezca funciones avanzadas disponibles mediante una suscripción. También se podría contemplar la opción de establecer una licencia para uso comercial.

Viabilidad legal

Protección de datos

Este proyecto no utiliza directamente datos personales de usuarios, pero dado que la herramienta NFStream procesa flujos de red para detectar

ataques DoS, el sistema interactúa con direcciones IP y puertos. Al tratarse de un entorno de pruebas controlado para la detección de ataques volumétricos sin almacenamiento persistente ni identificación de datos personales de terceros, el proyecto cumple con los lineamientos del Reglamento General de Protección de Datos (RGPD).

Licencias de Software

Para el desarrollo de este proyecto se han utilizado diversas herramientas y librerías de software, cada una con su respectiva licencia.

Librería	Versión	Licencia
Flask	3.0.0	BSD License
Flask-SQLAlchemy	3.1.1	BSD License
Jinja2	3.1.6	BSD License
MarkupSafe	3.0.3	BSD-3-Clause
PyNaCl	1.6.2	Apache Software License
Pygments	2.20.0	BSD-2-Clause
SQLAlchemy	2.0.50	MIT
Werkzeug	3.0.1	BSD License
annotated-types	0.7.0	MIT License
bcrypt	5.0.0	Apache Software License
blinker	1.9.0	MIT License
certifi	2026.5.20	Mozilla Public License 2.0 (MPL 2.0)
cffi	2.0.0	MIT
charset-normalizer	3.4.7	MIT
click	8.4.1	BSD-3-Clause
contourpy	1.3.2	BSD License
cryptography	48.0.0	Apache-2.0 OR BSD-3-Clause
cycler	0.12.1	BSD License
exceptiongroup	1.3.1	MIT License
fonttools	4.63.0	MIT

continúa en la página siguiente

continúa desde la página anterior

greenlet	3.5.1	MIT AND PSF-2.0
gunicorn	21.2.0	MIT License
idna	3.16	BSD-3-Clause
iniconfig	2.3.0	MIT
invoke	3.0.3	BSD-2-Clause
itsdangerous	2.2.0	BSD License
joblib	1.3.2	BSD License
kiwisolver	1.5.0	BSD License
matplotlib	3.8.3	Python Software Foundation License
numpy	1.26.4	BSD License
packaging	26.2	Apache-2.0 OR BSD-2-Clause
pandas	2.2.1	BSD License
paramiko	5.0.0	LGPL-2.1
pillow	12.2.0	MIT-CMU
pluggy	1.6.0	MIT License
psutil	5.9.8	BSD License
pycparser	3.0	BSD-3-Clause
pydantic	2.6.1	MIT
pydantic_core	2.16.2	MIT License
pyparsing	3.3.2	MIT
pytest	9.0.3	MIT
pytest-flask	1.3.0	MIT License
python-dateutil	2.9.0.post0	Apache Software License; BSD License
python-dotenv	1.0.1	BSD License
pytz	2026.2	MIT License
requests	2.31.0	Apache Software License
scikit-learn	1.4.2	BSD License
scipy	1.15.3	BSD License
seaborn	0.13.2	BSD License
six	1.17.0	MIT License
tensorboard	2.20.0	Apache Software License

continúa en la página siguiente

continúa desde la página anterior

tensorboard-data-server	0.7.2	Apache Software License
tensorflow	2.20.0	Apache Software License
threadpoolctl	3.6.0	BSD License
typing_extensions	4.15.0	PSF-2.0
tzdata	2026.2	Apache-2.0
urllib3	2.7.0	MIT

Tabla A.5: Licencias de librerías

Herramienta	Versión	Licencia
Arduino IDE	2.3.5	AGPL-3.0
Docker	4.61.0	Apache 2.0
Git	2.51.2.windows.1	GPLv2
GitHub	N/A	Propietaria
LaTeX	2024	LPPL
Overleaf	N/A	Propietaria
Python	3.10.20	PSFL
Visual Studio Code	1.122.1	MIT

Tabla A.6: Licencias de herramientas

Debido a la alta compatibilidad de estas licencias, se ha optado por escoger la licencia **MIT** para este proyecto, lo que permite que el código resultante del simulador y del microcontrolador pueda distribuirse libremente, siempre que se mantengan los reconocimientos de autoría originales.

Licencias de Recursos

Para el desarrollo de este proyecto se han utilizado diversos recursos, cada uno con su respectiva licencia.

Recurso	Licencia
Dataset NF-ToN-IoT	CC BY-NC-SA 4.0
Iconos Flaticon - Vectorslab	Propia

Tabla A.7: Licencias de recursos

El conjunto de datos empleado está distribuido bajo la licencia Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC

BY-NC-SA 4.0), por lo que cualquier modelo entrenado a partir de dicho conjunto de datos queda sujeto a las restricciones establecidas por esta licencia, lo que impide su utilización con fines comerciales.

Respecto a la documentación desarrollada para el proyecto, todas las imágenes incluidas han sido de creación propia. No obstante, algunas de ellas incorporan iconos creados por Vectorslab y distribuidos a través de la plataforma Flaticon. Dado que estos recursos permiten su uso comercial siempre que se mantenga la atribución correspondiente, su inclusión resulta compatible con la documentación generada.

Teniendo en cuenta las licencias de los materiales utilizados y la naturaleza de la documentación producida, se ha decidido publicar esta última bajo la licencia MIT. Esta licencia permite la libre utilización, distribución, modificación y adaptación del contenido, siempre que se conserve la atribución correspondiente.

Apéndice B

Especificación de Requisitos

B.1. Introducción

Esta sección proporciona una visión general del proyecto, detallando el propósito del software desarrollado, el valor que aporta, su público objetivo y el uso previsto del mismo.

Propósito del producto

El propósito de este software es proporcionar una plataforma web integral para la gestión y evaluación de modelos de Inteligencia Artificial orientados a la detección de ataques de Denegación de Servicio (DoS) en sistemas embebidos e IoT. La herramienta permite a los usuarios subir modelos preentrenados, registrar dispositivos y someter los modelos a pruebas de evaluación para comprobar su eficacia mediante métricas estandarizadas.

Valor del producto

Este producto aporta un gran valor al centralizar el testeo y la gestión de modelos de ciberseguridad en un entorno visual y controlado. Facilita la toma de decisiones al proporcionar matrices de confusión y métricas clave de rendimiento de forma instantánea, sin necesidad de que el usuario interactúe con código o terminales. Además, su arquitectura basada en contenedores asegura una alta replicabilidad.

Público objetivo

El sistema está dirigido a investigadores en ciberseguridad, ingenieros de datos y administradores de redes que busquen un entorno accesible y seguro para validar modelos de Machine Learning frente a trazas de red anómalas. También es ideal para propósitos académicos y demostrativos.

Uso previsto

La aplicación está diseñada para ser intuitiva. Permite el uso mediante cuentas registradas para la persistencia de datos, o mediante un modo *Invitado* para pruebas temporales. Los usuarios interactúan a través de un navegador web, cargan sus archivos de configuración y modelos, y analizan los resultados en paneles gráficos.

B.2. Objetivos generales

Este proyecto se ha desarrollado cumpliendo la siguiente serie de objetivos básicos:

- **Detectar ataques DoS** a microcontroladores utilizando computación **on-edge** y tratando de minimizar el tiempo de latencia para conseguir respuestas en **tiempo real**.
- Desarrollar una aplicación web segura que centralice la **detección de anomalías** y ataques DoS.
- Implementar un sistema de **gestión de modelos** de Machine Learning y **dispositivos IoT** vinculados a usuarios.
- Proveer un entorno de evaluación que devuelva **métricas precisas** sobre la eficacia de los modelos cargados.
- Garantizar la **privacidad e integridad** de los datos mediante políticas de retención, borrado en cascada y aislamiento de sesiones de invitados.

B.3. Catálogo de requisitos

Requisitos Funcionales

- **RF-1 Registro de usuarios:** se deben poder crear nuevas cuentas de usuario.

- **RF-2 Inicio de sesión:** el usuario debe poder iniciar sesión con una cuenta o entrar al sistema sin registro como usuario *invitado*.
- **RF-3 Gestionar cuenta:** el usuario debe poder visualizar y gestionar la configuración de su cuenta personal.
 - **RF-3.1 Modificar nombre de usuario:** el usuario debe poder cambiar el nombre asociado a su cuenta.
 - **RF-3.2 Modificar contraseña:** el usuario debe poder cambiar la contraseña de su cuenta.
 - **RF-3.3 Eliminar cuenta:** el usuario debe poder eliminar su cuenta de usuario del sistema.
- **RF-4 Gestionar dispositivos:** el usuario debe poder gestionar sus dispositivos IoT registrados en el sistema.
 - **RF-4.1 Añadir dispositivo:** el usuario debe poder registrar un nuevo dispositivo IoT en el sistema.
 - **RF-4.2 Eliminar dispositivo:** el usuario debe poder eliminar sus dispositivos registrados.
 - **RF-4.3 Mostrar dispositivos:** el usuario debe poder visualizar el listado de dispositivos registrados en el sistema.
- **RF-5 Gestionar modelos:** el usuario debe poder gestionar sus modelos IA cargados al sistema.
 - **RF-5.1 Añadir modelo:** el usuario debe poder subir un nuevo modelo de Machine Learning al sistema.
 - **RF-5.2 Eliminar modelo:** el usuario debe poder eliminar sus modelos y ficheros de configuración asociados.
 - **RF-5.3 Mostrar modelos:** el usuario debe poder visualizar el listado de modelos IA cargados en el sistema.
- **RF-6 Evaluación de modelos:** el usuario debe poder evaluar modelos IA cargados en el sistema con datos propios del sistema.
 - **RF-6.1 Seleccionar modelo:** el usuario debe poder seleccionar un modelo subido al sistema para su evaluación.
 - **RF-6.2 Evaluar modelo:** el sistema debe testear el rendimiento del modelo evaluándolo con datos de testeo del sistema.

- **RF-6.3 Mostrar métricas:** el sistema debe mostrar las métricas obtenidas en la evaluación del modelo y mostrarlas por pantalla al usuario.
- **RF-7 Gestionar laboratorio de detección:** el usuario debe poder iniciar un laboratorio de detección de anomalías.
 - **RF-7.1 Seleccionar artefactos:** el usuario debe poder seleccionar un modelo y un dispositivo registrado para la detección.
 - **RF-7.2 Comunicación:** el sistema debe establecer la comunicación SSH con el dispositivo IoT especificado.
 - **RF-7.3 Transferir archivos:** el sistema debe enviar los archivos de detección y modelo al dispositivo IoT.
 - **RF-7.4 Iniciar detección:** el usuario debe poder comenzar la detección de anomalías en el laboratorio de detecciones, y el sistema debe enviar la señal al dispositivo para ejecutar el script de detección.
 - **RF-7.5 Detener detección:** el usuario debe poder detener la detección, y el sistema debe enviar la señal al dispositivo para detener la ejecución.
 - **RF-7.6 Actualizar alarmas y gráfica:** el sistema debe recibir las alarmas de detección del dispositivo, mostrar el mensaje de aviso en la terminal del laboratorio y actualizar la gráfica.
 - **RF-7.7 Mostrar estadísticas:** el sistema debe recibir los resultados de la detección del dispositivo y mostrar al usuario las estadísticas obtenidas.
- **RF-8 Gestionar usuarios:** el administrador debe poder gestionar los usuarios del sistema.
 - **RF-8.1 Cambiar de rol:** el administrador debe poder cambiar el rol de un usuario del sistema.
 - **RF-8.2 Información de usuario:** el administrador debe poder visualizar la información de cuenta de los usuarios del sistema.
 - **RF-8.3 Mostrar usuarios:** el administrador debe poder ver el listado de usuarios del sistema.
 - **RF-8.4 Eliminar artefacto:** el administrador debe poder eliminar modelos y dispositivos de otros usuarios del sistema.
 - **RF-8.5 Eliminar usuario:** el administrador debe poder eliminar la cuenta de otro usuario del sistema.

- **RF-9 Gestión de roles:** el sistema debe ser capaz de distinguir entre tres tipos de usuarios según su rol: *Administrador*, *Usuario común* e *Invitado*.
- **RF-10 Persistencia:** el sistema debe eliminar automáticamente los modelos y dispositivos inactivos tras 40 días.

Requisitos No Funcionales

- **RNF-1 Seguridad:** Las contraseñas deben estar hasheadas mediante algoritmos seguros. El sistema debe validar estrictamente las extensiones de los archivos subidos. Las rutas de administración deben estar protegidas.
- **RNF-2 Usabilidad:** La interfaz debe proporcionar retroalimentación inmediata mediante mensajes y confirmaciones visuales antes de acciones destructivas.
- **RNF-3 Rendimiento y Fiabilidad:** El sistema debe procesar las evaluaciones de modelos en tiempos óptimos sin bloquear la interfaz. La base de datos debe mantener su integridad referencial en todo momento.
- **RNF-4 Mantenibilidad y escalabilidad:** El código debe estar bien estructurado y documentado de forma clara y concisa.
- **RNF-5 Disponibilidad:** El sistema debe estar disponible para su uso en navegadores compatibles con conexión a internet.

B.4. Especificación de requisitos

En este apartado se especifican los casos de uso del sistema y sus actores. En el sistema participan tres tipos de actores:

- **Administrador:** usuario con rol de administrador que tiene acceso a funcionalidades de gestión de usuarios.
- **Usuario común:** usuario con una cuenta registrada en base de datos.
- **Invitado:** usuario sin cuenta registrada. Los archivos cargados al sistema son temporales, se eliminan tras cerrar sesión.

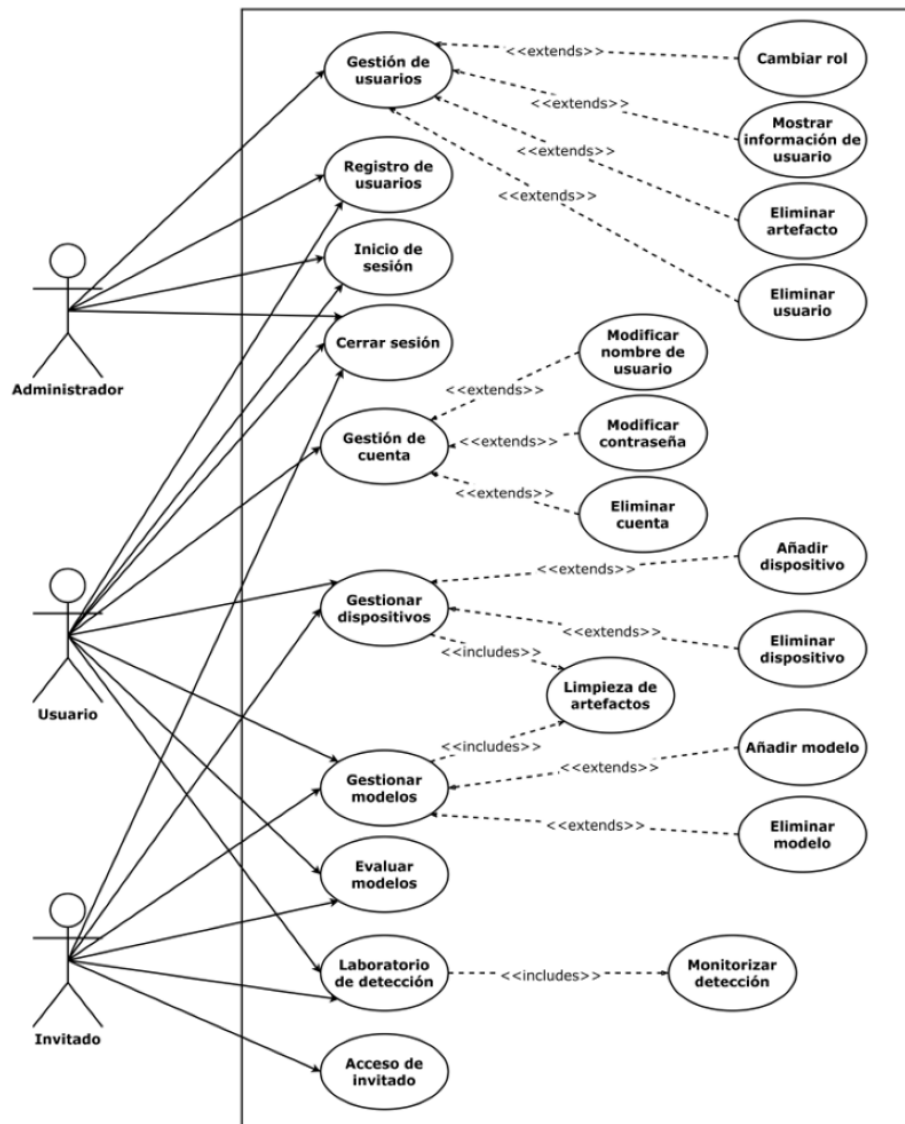


Figura B.1: Diagrama de casos de uso

CU-1	Registro de usuarios
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-1
Descripción	Permite a un nuevo usuario crear una cuenta en el sistema.
Precondición	El usuario tiene acceso a la página web y no tiene una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de registro. 2. Introduce un nombre de usuario y contraseña válidos. 3. El sistema valida los datos y registra al usuario en la base de datos.
Postcondición	El usuario queda registrado.
Excepciones	El nombre de usuario ya existe en el sistema. Los datos introducidos no cumplen los requisitos de formato.
Importancia	Alta

Tabla B.1: CU-1 Registro de usuarios.

CU-2	Inicio de sesión
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-2, RF-9
Descripción	Permite a usuarios registrados iniciar sesión en el sistema.
Precondición	El usuario está registrado en la base de datos y no tiene una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de inicio de sesión. 2. Introduce sus credenciales. 3. El sistema valida las credenciales y genera el token de sesión según su rol.
Postcondición	El usuario es redirigido a su panel de control personal.
Excepciones	Las credenciales son incorrectas.
Importancia	Alta

Tabla B.2: CU-2 Inicio de sesión.

CU-3	Acceso de invitado
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-2, RF-9
Descripción	Permite a usuarios acceder al sistema sin tener una cuenta registrada.
Precondición	El usuario tiene acceso a la página web y no tiene una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de inicio. 2. Accede al sistema como invitado. 3. El sistema genera una sesión temporal con el rol de Invitado.
Postcondición	El usuario es redirigido al panel principal.
Excepciones	-
Importancia	Media

Tabla B.3: CU-3 Acceso de invitado.

CU-4	Gestión de cuenta
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-3
Descripción	Permite a usuarios gestionar su cuenta.
Precondición	El usuario tiene una cuenta registrada y una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de su cuenta. 2. El sistema muestra la información del usuario.
Postcondición	Se muestra la configuración de cuenta actual.
Excepciones	-
Importancia	Alta

Tabla B.4: CU-4 Gestión de cuenta.

CU-5	Modificar nombre de usuario
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-3.1
Descripción	Permite a usuarios modificar el nombre asociado a su cuenta.
Precondición	El usuario tiene una cuenta registrada y una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de su cuenta. 2. Introduce un nuevo nombre de usuario, 3. El sistema valida los datos y actualiza la base de datos.
Postcondición	La configuración de cuenta se actualiza.
Excepciones	El nuevo nombre de usuario está en uso.
Importancia	Alta

Tabla B.5: CU-5 Modificar nombre de usuario.

CU-6	Modificar contraseña
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-3.2
Descripción	Permite a usuarios modificar su contraseña.
Precondición	El usuario tiene una cuenta registrada y una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de su cuenta. 2. Introduce una nueva contraseña, 3. Introduce de nuevo la contraseña por seguridad. 4. El sistema valida los datos y actualiza la base de datos.
Postcondición	La configuración de cuenta se actualiza.
Excepciones	La nueva contraseña no cumple los requisitos de seguridad.
Importancia	Alta

Tabla B.6: CU-6 Modificar contraseña.

CU-7	Eliminar cuenta
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-3.3
Descripción	Permite a usuarios eliminar su cuenta del sistema.
Precondición	El usuario tiene una cuenta registrada y una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de su cuenta. 2. El usuario selecciona la opción de eliminar cuenta. 3. El sistema solicita confirmación de la acción. 4. El usuario confirma y el sistema borra la cuenta y sus datos.
Postcondición	La cuenta se elimina y la sesión finaliza.
Excepciones	-
Importancia	Alta

Tabla B.7: CU-7 Eliminar cuenta.

CU-8	Cerrar sesión
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-9
Descripción	Permite a usuarios e invitados cerrar sesión del sistema.
Precondición	El usuario tiene una una sesión de usuario o invitado iniciada.
Acciones	1. El usuario selecciona la opción de cerrar sesión. 2. El sistema comprueba el rol del usuario. 3. Si es invitado: a) Elimina los archivos y dispositivos temporales de la sesión. b) Finaliza la sesión. c) El usuario es redirigido a la página principal. 4. Si no es invitado: a) Finaliza la sesión. b) El usuario es redirigido a la página principal.
Postcondición	La sesión finaliza.
Excepciones	-
Importancia	Alta

Tabla B.8: CU-8 Cerrar sesión.

CU-9	Gestionar dispositivos
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-4, RF-4.3
Descripción	Permite a usuarios gestionar los dispositivos IoT registrados en el sistema.
Precondición	El usuario tiene una una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de dispositivos. 2. El sistema muestra los dispositivos registrados.
Postcondición	Se muestran los dispositivos IoT registrados.
Excepciones	-
Importancia	Alta

Tabla B.9: CU-9 Gestionar dispositivos.

CU-10	Añadir dispositivo
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-4.1
Descripción	Permite a usuarios registrar nuevos dispositivos IoT.
Precondición	El usuario tiene una una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de dispositivos. 2. El usuario selecciona la opción de registro de un nuevo dispositivo. 3. El usuario introduce los datos del dispositivo (nombre e IP). 4. El sistema registra el dispositivo en la base de datos.
Postcondición	Se registra el nuevo dispositivo. Se muestra la lista de dispositivos actualizada.
Excepciones	-
Importancia	Alta

Tabla B.10: CU-10 Añadir dispositivo.

CU-11	Eliminar dispositivo
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-4.2
Descripción	Permite a usuarios eliminar dispositivos IoT registrados.
Precondición	El usuario tiene una una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de dispositivos. 2. El usuario selecciona la opción de eliminar un dispositivo del listado. 3. El sistema elimina el dispositivo de la base de datos.
Postcondición	Se elimina el dispositivo. Se muestra la lista de dispositivos actualizada.
Excepciones	-
Importancia	Alta

Tabla B.11: CU-11 Eliminar dispositivo.

CU-12	Gestionar modelos
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-5
Descripción	Permite a usuarios gestionar modelos IA registrados en el sistema.
Precondición	El usuario tiene una una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de modelos. 2. El sistema muestra los modelos registrados.
Postcondición	Se muestran los modelos IA registrados.
Excepciones	-
Importancia	Alta

Tabla B.12: CU-12 Gestionar modelos.

CU-13	Añadir modelo
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-5.1
Descripción	Permite a usuarios subir nuevos modelos IA.
Precondición	El usuario tiene una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de modelos. 2. El usuario selecciona la opción de carga de un nuevo modelo. 3. El usuario selecciona el archivo de modelo y de configuración. 4. El sistema registra ambos archivos en la base de datos.
Postcondición	Se registra el nuevo modelo. Se muestra la lista de modelos actualizada.
Excepciones	Formato de archivo no válido. Archivo corrupto durante la subida.
Importancia	Alta

Tabla B.13: CU-13 Añadir modelo.

CU-14	Eliminar modelo
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-5.2
Descripción	Permite a usuarios eliminar modelos IA registrados.
Precondición	El usuario tiene una una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de modelos. 2. El usuario selecciona la opción de eliminar un modelo del listado. 3. El sistema elimina el modelo y su archivo de configuración asociado de la base de datos.
Postcondición	Se elimina el modelo. Se muestra la lista de modelos actualizada.
Excepciones	-
Importancia	Alta

Tabla B.14: CU-14 Eliminar modelo.

CU-15	Limpieza de artefactos
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-10
Descripción	Elimina archivos en desuso del sistema.
Precondición	Un usuario ha iniciado sesión.
Acciones	1. El usuario inicia sesión. 2. El sistema comprueba la fecha de último uso de los artefactos (modelos y dispositivos) del usuario. 3. El sistema elimina de la base de datos aquellos cuyo último uso supere los 40 días. 4. El sistema muestra un mensaje informando al usuario de la limpieza de archivos.
Postcondición	Se elimina el modelo. Se muestra la lista de modelos actualizada.
Excepciones	-
Importancia	Alta

Tabla B.15: CU-15 Limpieza de artefactos.

CU-16	Evaluar modelos
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-6, RF-6.1, RF-6.2, RF-6.3
Descripción	Permite a usuarios evaluar modelos.
Precondición	El usuario tiene una sesión iniciada.
Acciones	1. El usuario accede a la pantalla de evaluación. 2. El usuario selecciona un modelo de los disponibles y selecciona la opción de evaluación. 3. El sistema carga los archivos de modelo y configuración seleccionados. 4. El sistema ejecuta el script de evaluación con los datos de testeo propios. 5. El sistema registra los resultados obtenidos. 6. El sistema muestra estadísticas al usuario: ▪ Matriz de confusión ▪ Cantidad de flujos de evaluación ▪ <i>Accuracy</i> ▪ <i>F1-score</i> ▪ <i>Precision</i> ▪ <i>Recall</i>
Postcondición	El usuario visualiza los resultados de rendimiento del modelo.
Excepciones	No se selecciona un modelo. No se encuentran los archivos de modelo o configuración. No se encuentra el dataset de testeo. Ocurre un error durante la evaluación.
Importancia	Media

Tabla B.16: CU-16 Evaluar modelos.

CU-17	Laboratorio de detección
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-7, RF-7.1, RF-7.2, RF-7.3
Descripción	Permite a usuarios preparar un entorno de detección de anomalías.
Precondición	El usuario tiene una sesión iniciada y tiene al menos un modelo y dispositivo registrado.
Acciones	1. El usuario accede a la pantalla de detección. 2. El usuario selecciona un modelo y dispositivo de los disponibles. 3. El sistema solicita las credenciales SSH del dispositivo. 4. El sistema inicia la comunicación con el dispositivo. 5. El sistema transfiere los archivos de modelo y configuración, y el script de detección al dispositivo. 6. El usuario es redirigido al laboratorio de detección.
Postcondición	El sistema está conectado al dispositivo.
Excepciones	No se selecciona un modelo/dispositivo. No se encuentran los archivos de modelo o configuración. No se puede conectar con el dispositivo. Credenciales SSH incorrectas.
Importancia	Alta

Tabla B.17: CU-17 Laboratorio de detección.

CU-18	Monitorizar detección
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-7, RF-7.4, RF-7.5, RF-7.6, RF-7.7
Descripción	Permite a usuarios iniciar la detección de anomalías.
Precondición	El usuario está en el laboratorio de detección y se ha comenzado la comunicación con el dispositivo.
Acciones	1. El usuario inicia la detección. 2. El sistema recibe y muestra las alarmas actualizando la gráfica de la interfaz. 3. El usuario ordena detener la detección. 4. El sistema manda la señal de interrupción por SSH. 5. Se compilan y muestran las estadísticas finales.
Postcondición	Finaliza la detección. Se muestran las estadísticas.
Excepciones	Pérdida de conexión con el dispositivo durante la ejecución.
Importancia	Alta

Tabla B.18: CU-18 Monitorizar detección.

CU-19	Gestión de usuarios
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-8, RF-8.3
Descripción	Permite a administradores gestionar los usuarios del sistema.
Precondición	El usuario ha iniciado sesión como administrador.
Acciones	1. El administrador accede a la pantalla de gestión. 2. El sistema muestra el listado de usuarios registrados.
Postcondición	Se muestra el listado de usuarios.
Excepciones	-
Importancia	Alta

Tabla B.19: CU-19 Gestión de usuarios.

CU-20	Cambiar rol
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-8.1
Descripción	Permite a administradores cambiar el rol de un usuario.
Precondición	El usuario ha iniciado sesión como administrador.
Acciones	1. El administrador accede a la pantalla de gestión. 2. El administrador selecciona la opción de cambio de rol de un usuario. 3. El sistema invierte el rol actual del usuario y guarda los cambios en la base de datos.
Postcondición	Se actualiza el rol del usuario. Se muestra el listado de usuarios actualizado.
Excepciones	-
Importancia	Alta

Tabla B.20: CU-20 Cambiar rol.

CU-21	Mostrar información de usuario
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-8.2
Descripción	Permite a administradores ver la información de cuenta de un usuario.
Precondición	El usuario ha iniciado sesión como administrador.
Acciones	1. El administrador accede a la pantalla de gestión. 2. El administrador selecciona un usuario. 3. El sistema muestra la información de cuenta del usuario seleccionado.
Postcondición	Se muestra la información del usuario.
Excepciones	-
Importancia	Alta

Tabla B.21: CU-21 Mostrar información de usuario.

CU-22	Eliminar artefacto
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-8.4
Descripción	Permite a administradores eliminar modelos y dispositivos de un usuario.
Precondición	El usuario ha iniciado sesión como administrador y ha seleccionado un usuario.
Acciones	1. El administrador selecciona la opción de borrado de un modelo/dispositivo. 2. El sistema pide confirmación para eliminar. 3. El sistema elimina el modelo (y su archivo de configuración)/dispositivo de la base de datos.
Postcondición	Se elimina el modelo/dispositivo. Se muestra la información del usuario actualizada.
Excepciones	-
Importancia	Alta

Tabla B.22: CU-22 Eliminar artefacto.

CU-23	Eliminar usuario
Versión	1.0
Autor	Sara Abejón Pérez
Requisitos asociados	RF-8.4
Descripción	Permite a administradores eliminar un usuario.
Precondición	El usuario ha iniciado sesión como administrador.
Acciones	1. El administrador accede a la pantalla de gestión. 2. El administrador selecciona la opción de borrado de un usuario. 3. El sistema pide confirmación para eliminar. 4. El sistema elimina la cuenta y los datos del usuario de la base de datos.
Postcondición	Se elimina el usuario y sus archivos. Se muestra el listado de usuarios actualizado.
Excepciones	-
Importancia	Alta

Tabla B.23: CU-23 Eliminar usuario.

Apéndice C

Especificación de diseño

C.1. Introducción

La especificación de diseño detalla la arquitectura interna y la estructura de datos que componen el software desarrollado. Esta sección actúa como el puente entre los requisitos funcionales establecidos en la fase de análisis y su posterior implementación técnica. Se describen los diferentes aspectos del diseño de esta aplicación, incluyendo el diseño de la base de datos relacional, la arquitectura del sistema y el flujo de los procedimientos principales.

C.2. Diseño de datos

Modelo Entidad-Relación

Para representar gráficamente las relaciones entre los datos, se ha creado un diagrama E-R (ver figura C.1).

Entidades

- **Entidad Usuario (User):** Es la entidad principal del sistema. Puede tener dos roles principales diferenciados por un atributo booleano (Administrador o Usuario estándar).
- **Entidad Modelo (Modelo):** Representa los modelos de Inteligencia Artificial.
- **Entidad Archivo de Configuración (File):** Representa los archivos de características asociados a un modelo.

- **Entidad Dispositivo (Dispositivo):** Representa los nodos IoT donde se ejecutan los scripts de detección.

Relaciones

- **Relación Sube (1:N):** Un Usuario puede subir y poseer múltiples Modelos (1 a N). Un Modelo pertenece a un único Usuario (1 a 1). Esta relación implementa un borrado en cascada (si el usuario se elimina, sus modelos también).
- **Relación Registra (1:N):** Un Usuario puede registrar múltiples Dispositivos (1 a N). Un Dispositivo pertenece a un único Usuario (1 a 1). También cuenta con borrado en cascada.
- **Relación Contiene (1:N):** Un Modelo puede tener asociados varios archivos adicionales de configuración File (1 a N), aunque la cardinalidad habitual es uno a uno.

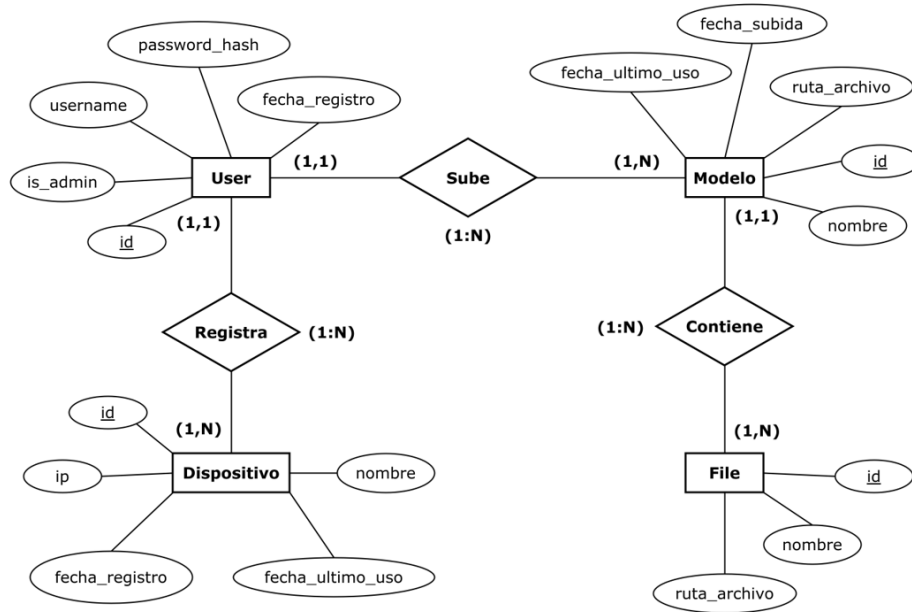


Figura C.1: Diagrama Modelo Entidad-Relación

Entidades efímeras

Para satisfacer el requisito funcional de acceso sin registro (**RF-1.3**), garantizando a la vez que la base de datos principal no se sature con

información residual, el sistema implementa una arquitectura paralela de entidades efímeras que no pertenecen al Modelo Entidad-Relación, pero son importantes en el desarrollo y funcionamiento del proyecto.

A diferencia de los usuarios registrados, los usuarios que acceden en modo *Invitado* utilizan una abstracción basada en la memoria de sesión del servidor. Para ello, se han diseñado clases de Python que no heredan de `db.Model`:

- **Invitado:** Estructura de datos principal que simula a un usuario temporal. Genera un ID de sesión único y gestiona el ciclo de vida del invitado.
- **ModeloInvitado:** Simula el comportamiento de la clase *Modelo*. En lugar de guardar una ruta física permanente, almacena temporalmente los archivos en el directorio de cachés temporales y vincula sus metadatos a la lista de la sesión.
- **DispositivoInvitado:** Almacena los parámetros de red de los dispositivos directamente en el diccionario de la sesión activa del usuario.

Estas clases no tienen la integridad referencial de una base de datos relacional, por lo que la gestión de recursos se realiza en el nivel del controlador. Cuando la sesión del invitado finaliza, el controlador de autenticación ejecuta un procedimiento de *recolección de basura* explícito. Este procedimiento recorre las listas de `ModeloInvitado`, elimina físicamente los archivos del directorio temporal asociado a ese ID de sesión y limpia el objeto de la RAM, asegurando una huella cero en el servidor.

Modelo Relacional

Partiendo del modelo Entidad-Relación anterior, se ha creado el diagrama relacional:

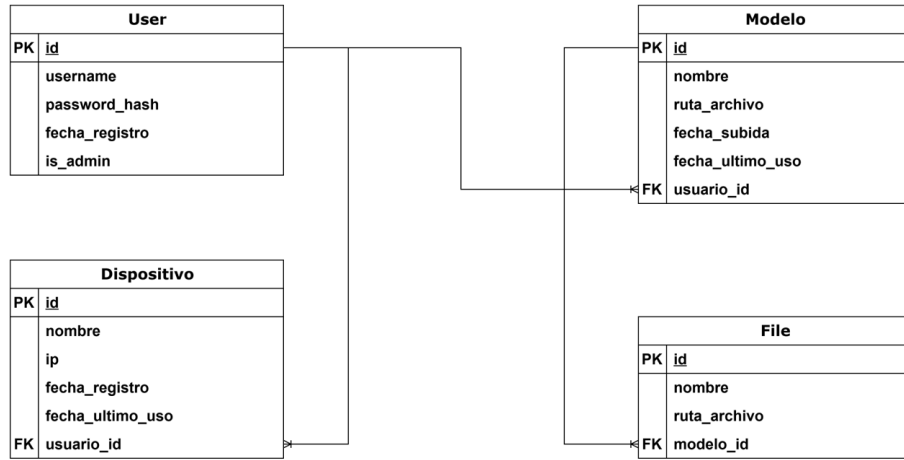


Figura C.2: Diagrama relacional

Diccionario de datos

Para cada tabla de la base de datos, se ha creado un diccionario de datos que contiene la información correspondiente a los tipos de entidades con las que se trabaja.

Nombre	Tipo	Columna	Descripción
id	INTEGER	PK	Identificador único del usuario.
username	VARCHAR(50)	UNIQUE NOT NULL	Nombre de cuenta del usuario.
password_hash	VARCHAR(256)	NOT NULL	Contraseña del usuario cifrada.
fecha_registro	DATETIME	NOT NULL	Fecha de creación de la cuenta.
is_admin	BOOLEAN	NOT NULL	Indica si el usuario tiene privilegios de administrador. Por defecto es False.

Tabla C.1: Diccionario de datos. Tabla correspondiente a la clase **User**.

Nombre	Tipo	Columna	Descripción
id	INTEGER	PK	Identificador único del modelo.
nombre	VARCHAR(150)	NOT NULL	Nombre asignado al modelo.
ruta_archivo	VARCHAR(255)	NOT NULL	Ruta física donde se almacena el archivo dentro del servidor.
usuario_id	INTEGER	FK(usuarios.id)	Identificador del usuario propietario del modelo.
fecha_subida	DATETIME	NOT NULL	Fecha en la que se subió el modelo.
fecha_ultimo_datos	DATETIME	NOT NULL	Fecha de la última vez que el modelo fue evaluado o desplegado.

Tabla C.2: Diccionario de datos. Tabla correspondiente a la clase `Modelo`.

Nombre	Tipo	Columna	Descripción
id	INTEGER	PK	Identificador único del dispositivo IoT.
nombre	VARCHAR(150)	NOT NULL	Nombre representativo del dispositivo.
ip	VARCHAR(50)	NOT NULL	Dirección IP del dispositivo.
usuario_id	INTEGER	FK(usuarios.id)	Identificador del usuario que ha registrado el dispositivo.
fecha_registro	DATETIME	NOT NULL	Fecha de registro del dispositivo.
fecha_ultimo_datos	DATETIME	NOT NULL	Fecha de la última comunicación exitosa con el dispositivo.

Tabla C.3: Diccionario de datos. Tabla correspondiente a la clase `Dispositivo`

Nombre	Tipo	Columna	Descripción
id	INTEGER	PK	Identificador único del archivo de configuración.
nombre	VARCHAR(150)	NOT NULL	Nombre del archivo.
ruta_archivo	VARCHAR(255)	NOT NULL	Ruta física de almacenamiento del archivo en el volumen del servidor.
modelo_id	INTEGER	FK(modelos.id)	Referencia al modelo al que complementa.

Tabla C.4: Diccionario de datos. Tabla correspondiente a la clase `File`.

C.3. Diseño arquitectónico

Para garantizar que el software sea robusto, escalable y fácilmente mantenible, el proyecto se ha dividido en dos niveles arquitectónicos fundamentales: la arquitectura interna del software, basada en el patrón de diseño Modelo-Vista-Controlador, y la arquitectura global del sistema y despliegue, basada en contenedores.

Esta separación de responsabilidades permite aislar la lógica de negocio de la interfaz de usuario, y a su vez, independizar la aplicación del hardware subyacente del servidor.

Arquitectura de Software: Patrón MVC (Modelo-Vista-Controlador)

La aplicación está construida sobre el framework web Flask, el cual adapta y fomenta el uso del patrón de arquitectura MVC. Este paradigma divide la aplicación en tres componentes lógicos interconectados:

- **Modelo (Model):** Representa la capa de acceso y gestión de datos. En este proyecto, se implementa a través del ORM (Object-Relational Mapping) Flask-SQLAlchemy. El modelo actúa como intermediario entre la lógica de Python y la base de datos relacional SQLite, abstrayendo las consultas SQL. Aquí residen las clases que definen la estructura del sistema: `User`, `Modelo`, `Dispositivo` y `File`.
- **Vista (View):** Es la capa de presentación encargada de la interfaz gráfica de usuario. Se ha desarrollado utilizando el motor de plantillas

Jinja2, combinado con HTML5, CSS3 y JavaScript para la interactividad asíncrona. La Vista no contiene lógica de negocio, limitándose a renderizar de forma segura la información que recibe del Controlador.

- **Controlador (Controller):** Es el cerebro de la aplicación. Se ha estructurado de forma modular utilizando Blueprints de Flask. El Controlador intercepta las peticiones HTTP del usuario, verifica los controles de acceso y sesión, interroga o actualiza el Modelo según sea necesario, y finalmente procesa los datos para inyectarlos en la Vista adecuada.

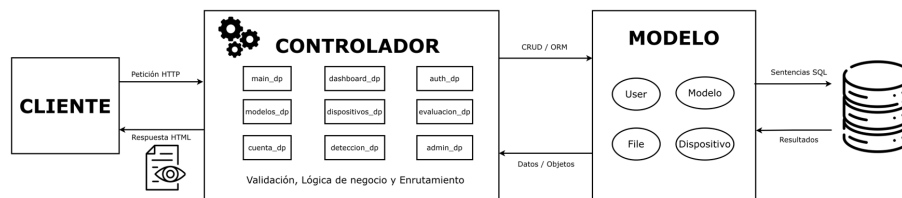


Figura C.3: Arquitectura MVC del Software.

Arquitectura del Sistema y Despliegue (Docker)

Para resolver los problemas de incompatibilidad de entornos y dependencias, la arquitectura global del sistema se ha diseñado bajo un enfoque de **Contenedorización** mediante Docker.

En lugar de instalar las dependencias directamente en el sistema operativo del host, la plataforma entera se encapsula en una imagen de Docker. Esta arquitectura se orquesta a través del archivo `docker-compose.yml`, definiendo cómo interactúa el contenedor con el exterior y cómo persisten los datos.

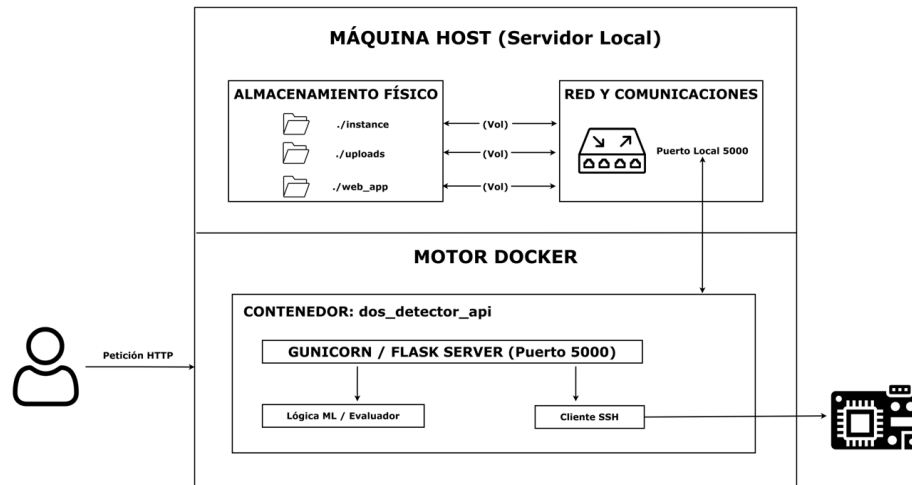


Figura C.4: Arquitectura por contenedores.

Los elementos clave de la arquitectura vista en la Figura C.4 son:

- **Contenedor API (`dos_detector_api`):** Es el entorno aislado que ejecuta el servidor de la aplicación web.
- **Gestión de Red y Puertos (Port Mapping):** El contenedor expone su tráfico a través del puerto 5000, el cual se mapea directamente al puerto 5000 de la máquina Host, permitiendo el acceso de los clientes web y la salida de conexiones SSH (Puerto 22) hacia los nodos IoT.
- **Persistencia de Datos (Volúmenes):** Para que la información no se destruya al apagar el contenedor, se ha implementado un sistema de volúmenes sincronizados en tiempo real:
 - **Volumen de Código (`/app`):** Sincroniza el código fuente del Host con el contenedor, permitiendo el desarrollo en caliente.
 - **Volumen de Base de Datos (`/instance`):** Asegura que el archivo `app.db` resida físicamente en el disco del Host.
 - **Volumen de Modelos (`/uploads`):** Protege el almacenamiento de los archivos `.pkl` y `.npz` subidos por los usuarios para que persistan entre reinicios.

C.4. Diseño procedimental

En esta sección se describe el procedimiento central de la plataforma: la detección de ataques DoS en tiempo real, abarcando desde la interacción del usuario en la aplicación web hasta la ejecución remota en el dispositivo IoT y el manejo de posibles fallos.

Secuencia de detección en tiempo real

El proceso de despliegue y ejecución del laboratorio de detección involucra tres componentes principales interactuando a lo largo del tiempo: el usuario, la plataforma web y el dispositivo IoT.

El intercambio de mensajes se produce de la siguiente manera:

1. **Inicialización:** El usuario selecciona en la interfaz web el modelo predictivo y el dispositivo objetivo, y lanza la petición de inicio.
2. **Establecimiento de canales:** La plataforma establece un túnel seguro SSH contra el dispositivo. Una vez autenticado, utiliza el protocolo SFTP para transferir los artefactos necesarios.
3. **Ejecución remota:** El servidor envía el comando a través de la terminal remota para arrancar el script. El dispositivo comienza a capturar y analizar el tráfico de red de forma autónoma.
4. **Bucle de monitorización y alertas:** Mientras el dispositivo procesa los flujos, si el modelo clasifica un paquete como malicioso, el dispositivo interrumpe su función y envía una petición asíncrona a la plataforma. El servidor procesa la alarma y actualiza dinámicamente la interfaz gráfica del usuario.
5. **Finalización:** Cuando el usuario decide abortar la monitorización, la plataforma envía una señal de interrupción por el canal SSH. El dispositivo detiene la captura, compila las estadísticas finales de la sesión y las devuelve al servidor antes de que este cierre la conexión y muestre el informe definitivo al usuario.

La figura muestra el diagrama de secuencia del proceso.

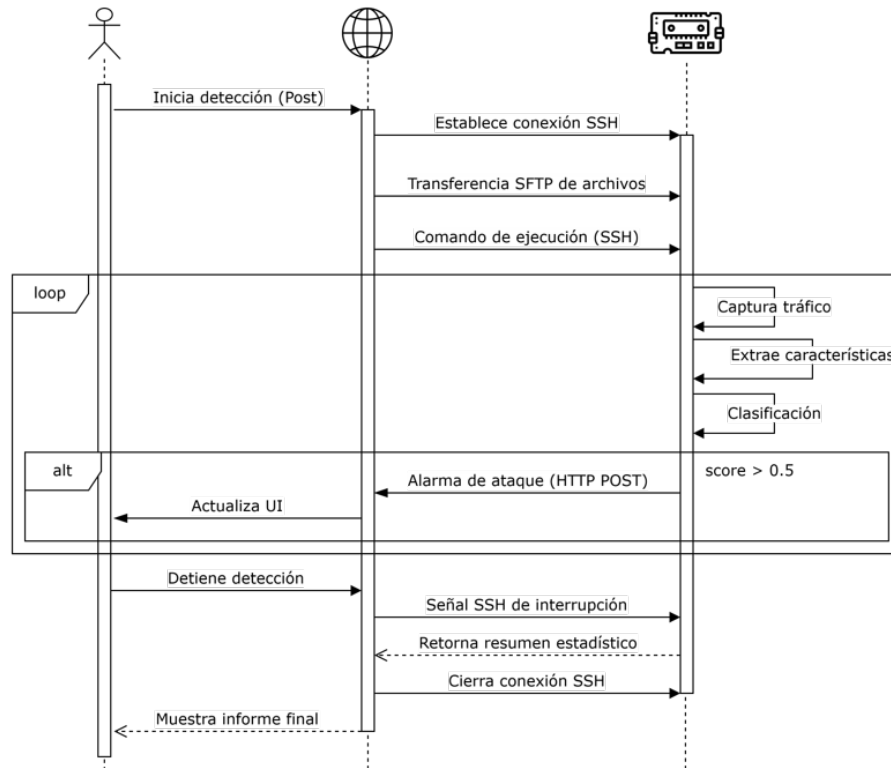


Figura C.5: Diagrama de secuencia de la detección.

Manejo de errores y excepciones

La aplicación y la secuencia de comunicación incluyen mecanismos de control para manejar excepciones en cada etapa crítica:

- **Fallo en el establecimiento (Timeout SSH):** Si en el paso 2 el dispositivo está apagado, fuera de la red, o la dirección IP registrada es incorrecta, la librería *Paramiko* lanzará una excepción. El servidor aborta el flujo antes de transferir datos y devuelve una respuesta al usuario indicando que no se ha podido establecer la conexión.
- **Incompatibilidad de artefactos:** Si los archivos de modelo o configuración transferidos en el paso 3 están corruptos o las características del modelo no coinciden con las extraídas por el dispositivo, el script remoto fallará al iniciar. El servidor web monitoriza el canal de error estándar del SSH; al detectar una excepción en remoto, cierra el túnel y alerta al usuario sobre la incompatibilidad del modelo.

- **Pérdida de conexión en caliente (Drop de red):** Durante el bucle de detección, es posible que el microcontrolador pierda la conexión Wi-Fi temporalmente. Si los webhooks de las alarmas fallan consecutivamente, o si el servidor web pierde el pulso de la sesión SSH, el sistema da por finalizada la monitorización de forma forzosa. La interfaz del usuario se actualizará mostrando una notificación de desconexión prematura y renderizará las estadísticas recuperadas hasta el último punto de control válido.
- **Fallo de privilegios en la captura local:** Si la herramienta NFS-tream en el microcontrolador no tiene los permisos suficientes para abrir el socket crudo en la interfaz de red, el script captura la excepción nativa localmente, enviando un código de error de vuelta al servidor web en lugar del resumen estadístico, abortando la ejecución de forma controlada.

Apéndice *D*

Documentación técnica de programación

D.1. Introducción

En este apéndice se documentan los aspectos técnicos fundamentales para la comprensión, mantenimiento y posible extensión del código fuente desarrollado en este proyecto. Se detalla la estructura completa de los directorios del repositorio, el entorno de desarrollo necesario, las instrucciones de compilación y ejecución, así como la batería de pruebas unitarias implementadas para garantizar la calidad y seguridad del software.

D.2. Estructura de directorios

El código fuente y los recursos del proyecto se encuentran en un repositorio público de GitHub bajo la licencia MIT. Está organizado de forma modular para separar la documentación, el código de los microcontroladores y la plataforma web. A continuación, se presenta la jerarquía de los directorios y archivos del proyecto:

- ***docs/***: Contiene todos los archivos relacionados con la documentación del proyecto.
 - ***diseño/***: Incluye dos archivos que recogen la versión inicial y teórica de la arquitectura y funcionalidad del sistema, utilizados como base durante los primeros sprints de implementación.

- **informe/**: Contiene los archivos PDF finales, correspondientes a la Memoria y los Anexos del proyecto.
- **dos_detector/**: Es el directorio principal de la aplicación web y su entorno de despliegue.
 - **docker-compose.yaml**: Archivo de orquestación de Docker para reproducir el sistema de forma automatizada.
 - **web_app/**: Contiene el código fuente íntegro de la plataforma
 - **app/**: Código central de la aplicación web (rutas, modelos, plantillas y lógica de Flask).
 - **data/**: Almacena el archivo CSV empleado para la evaluación de los modelos dentro del sistema.
 - **storage/**: Directorio destinado a albergar la base de datos persistente.
 - **test/**: Directorio que contiene las pruebas unitarias del sistema (detallado en la sección D.5).
 - **uploads/**: Volumen montado mediante Docker para almacenar de forma persistente los modelos subidos por los usuarios.
 - **Archivos de configuración**: *.dockerignore*, *Dockerfile* (construcción de la imagen), *requirements.txt* (dependencias de Python) y *run.py* (punto de entrada de la aplicación).
- **m5core/**: Contiene los códigos fuente correspondientes a las pruebas y experimentación de la fase inicial del proyecto.
- **m5llm/**: Directorio dedicado a la implementación del entorno de detección en tiempo real.
 - **codes/**: Archivos de código para la ejecución de modelos y la captura/detección en tiempo real. Incluye un subdirectorio con versiones anteriores.
 - **data/**: Contiene el archivo *columns_no_gen.txt* que especifica las características de los flujos de red que deben ser excluidas durante el entrenamiento del modelo para evitar el sobreajuste al entorno de laboratorio.
 - **models/**: Almacena los modelos entrenados, sus respectivos archivos de configuración (*features.npy*) y las matrices de confusión generadas.
 - **.gitignore**: Filtro de exclusión de archivos locales.

- **.gitignore:** Archivo de configuración global de Git.
- **LICENSE:** Archivo de licencia que indica el contenido de la licencia usada.
- **README.md:** Documento principal de presentación del repositorio, con información general del proyecto.

D.3. Manual del programador

Obtención del código fuente

El código fuente del proyecto completo se puede obtener, clonando el repositorio o bien descargándolo directamente desde GitHub¹.

Entorno de desarrollo

Para la reproducción completa del proyecto, se recomienda tener instaladas las siguientes herramientas, aunque se podría realizar con otras:

- Python ≥ 3.10
- Docker y Docker Desktop
- Git
- IDE (Visual Studio Code)

D.4. Compilación, instalación y ejecución del proyecto

Gracias a la contenedorización, el despliegue del sistema web se ha simplificado notablemente, abstrayendo las complejidades de configuración del servidor. Para compilar e instalar la plataforma, se deben seguir estos pasos:

1. Clonar el repositorio

```
git clone https://github.com/saraabejonperez/Detection-and-analysis-of-attacks-on-embedded-systems.git
```

¹<https://github.com/saraabejonperez/Detection-and-analysis-of-attacks-on-embedded-systems.git>

2. Situar en el directorio de la aplicación web
`cd dos_detector`
3. Contruir la imagen y levantar el evento
`docker-compose up -build -d`
4. Acceder a la plataforma. Una vez el contenedor esté en ejecución, la aplicación web estará disponible a través del navegador en `http://ip_host:5000`. Es necesario entrar a la aplicación con la ip de la máquina anfitriona para evitar errores de comunicación con el dispositivo.

D.5. Pruebas del sistema

Se ha implementado una suite de pruebas automatizadas que simulan el comportamiento del usuario y validan las restricciones del sistema interactuando con una base de datos temporal en memoria, asegurando que los tests no alteren el entorno de producción.

La configuración global de las pruebas se define en el archivo *conftest.py*, donde se instancian los accesorios necesarios: la creación de la aplicación en modo TESTING con una base de datos SQLite en RAM, un cliente virtual de pruebas y la inserción inicial de un usuario de prueba en la base de datos.

A continuación, se detallan los módulos de pruebas desarrollados:

Pruebas de Autenticación y Acceso

Valida la correcta disponibilidad de las rutas públicas y la gestión de la sesión de invitados.

- **test_pagina_inicio_carga_bien:** Verifica que el servidor responde correctamente a las peticiones en la raíz de la web y renderiza el contenido esperado.
- **test_acceso_invitado:** Simula el acceso mediante el modo invitado. Comprueba que el servidor ejecuta una redirección y que el usuario es capaz de acceder posteriormente al panel de control con un estado HTTP 200.

Pruebas de Seguridad y Subida de Archivos

Garantiza que el sistema bloquea intentos de inyección de código y archivos incompletos durante el registro de modelos.

- **test_subida_extension_invalida:** Inyecta archivos ficticios en la memoria RAM simulando un archivo no permitido. Valida que el servidor intercepta la petición, rechaza el formulario y devuelve un mensaje de error.
- **test_subida_falta_archivo:** Evalúa la robustez del formulario enviando únicamente el modelo predictivo, omitiendo el archivo de configuración. El sistema debe denegar la operación y devolver una advertencia.

Pruebas de Control de Acceso por Roles

Verifica la correcta implementación del Role-Based Access Control (RBAC) para proteger rutas sensibles.

- **test_acceso_admin_denegado_usuario_normal:** Simula la sesión de un usuario registrado sin privilegios intentando acceder por la fuerza a la ruta `/admin/usuarios`. La prueba corrobora que la vista es denegada, redirigiendo al usuario y mostrando un mensaje de alerta de permisos insuficientes.
- **test_acceso_admin_permitido:** Inyecta en la sesión los parámetros de un administrador y verifica que el acceso a la tabla de "Gestión de Usuarios" ^{es} autorizado exitosamente.

Pruebas de Integridad de la Base de Datos

Valida el comportamiento estructural y la integridad referencial del ORM de SQLAlchemy.

- **test_borrado_en_cascada:** Prueba el efecto dominó en la base de datos. Se asocia un modelo y un dispositivo falso a un usuario. Tras confirmar su persistencia, se elimina al usuario. La prueba verifica rigurosamente que las consultas posteriores devuelven 0, demostrando que los modelos y dispositivos huérfanos se han borrado de forma automática y segura.

Apéndice E

Documentación de usuario

- E.1. Introducción
- E.2. Requisitos de usuarios
- E.3. Instalación
- E.4. Manual del usuario

Anexo de sostenibilización curricular

F.1. Introducción

El desarrollo tecnológico actual, impulsado por la proliferación del Internet de las Cosas, plantea importantes retos no solo en el ámbito de la ciberseguridad, sino también en el de la sostenibilidad. Siguiendo las directrices trazadas por la Conferencia de Rectores de las Universidades Españolas (CRUE) para la introducción de la sostenibilidad en el currículum universitario, este apéndice expone una reflexión crítica sobre el impacto ambiental, social y económico de las tecnologías empleadas en este proyecto, así como las competencias sostenibles adquiridas durante su desarrollo.

Tradicionalmente, la seguridad perimetral y el análisis de intrusiones en redes IoT han dependido de infraestructuras centralizadas (Cloud Computing). Este enfoque, si bien es potente, conlleva un alto coste ecológico derivado del consumo energético necesario para la transmisión constante de grandes volúmenes de datos y el mantenimiento de servidores de alta disponibilidad. A través de este proyecto, he comprendido que la elección de la arquitectura tecnológica tiene un impacto directo en la huella de carbono digital. Al apostar por el Edge Computing, trasladando el procesamiento y la detección de ataques de Denegación de Servicio directamente al microcontrolador, se reduce drásticamente el consumo de ancho de banda y la energía asociada a la transmisión de datos en la red. Asimismo, se promueve una mayor resiliencia y privacidad, pilares fundamentales de un entorno digital ético y sostenible.

F.2. Competencias de sostenibilidad aplicadas

Durante el desarrollo del proyecto, se han trabajado e interiorizado las cuatro competencias transversales de sostenibilidad definidas por la CRUE:

SOS1: Competencia en el conocimiento y comprensión crítica de las relaciones entre las problemáticas ambientales, sociales y económicas.

He adquirido una visión holística sobre cómo los ciberataques, en particular los ataques DoS, no solo generan graves pérdidas económicas o interrupciones de servicios sociales críticos, sino que también representan un enorme desperdicio de recursos energéticos. Cuando una red es víctima de un ataque volumétrico, la infraestructura consume energía de manera ineficiente procesando "tráfico basura". Al desarrollar un sistema capaz de detectar estas anomalías en origen, se contribuye a la creación de infraestructuras digitales más eficientes y resilientes, minimizando el impacto económico y el gasto energético derivado de la saturación de los recursos de red y de los servidores.

SOS2: Competencia en la comprensión de las posibles vías y opciones para intervenir en el desarrollo sostenible.

A lo largo del proyecto, he evaluado diferentes alternativas tecnológicas buscando aquellas que representaran la vía de intervención más sostenible. La elección de un modelo de aprendizaje automático ligero, como el Random Forest, frente a alternativas de aprendizaje profundo computacionalmente más demandantes, responde a una decisión consciente para minimizar la huella energética. Esto ha permitido que la inferencia se pueda ejecutar en un dispositivo embebido con recursos limitados, optimizando el uso del hardware y demostrando que es posible intervenir en la seguridad de la red sin necesidad de desplegar costosas infraestructuras en la nube.

SOS3: Competencia en la aplicación de los principios del desarrollo sostenible en la prevención de impactos negativos sobre el medio natural y social.

Este principio se ha aplicado de manera directa en el diseño de la arquitectura y las reglas de negocio de la plataforma web. Se ha implementado una política automatizada de retención de datos que elimina los modelos de inteligencia artificial almacenados tras 40 días ininterrumpidos sin uso. Esta medida combate de forma activa la acumulación de archivos inactivos en servidores que generan un consumo energético continuo en los centros de datos. De esta forma, se previenen los impactos ecológicos negativos asociados al almacenamiento masivo innecesario, promoviendo un uso racional y responsable de los recursos de la nube.

SOS4: Competencia en la aplicación de principios éticos relacionados con los valores de la sostenibilidad en los comportamientos personales y profesionales.

La ética profesional en la ingeniería informática exige garantizar la privacidad y la protección de los datos de los usuarios. Al aislar el proceso de detección en el propio microcontrolador mediante Edge Computing, los flujos de red que procesa la herramienta NFStream no abandonan la red local para ser analizados por terceros. Esto fomenta una equidad social basada en el derecho a la privacidad. Además, este enfoque ético se ha reflejado en el uso de tecnologías y librerías de código abierto, promoviendo la accesibilidad del conocimiento y asegurando que las soluciones desarrolladas puedan ser replicadas por la comunidad sin barreras económicas.

F.3. Conclusiones

En conclusión, la integración de la sostenibilidad en la ingeniería de software y en la arquitectura de sistemas embebidos no es un añadido opcional, sino una necesidad intrínseca del desarrollo tecnológico moderno. Este proyecto me ha demostrado que es plenamente factible armonizar la ciberseguridad avanzada con el respeto al medio ambiente. Mediante la descentralización del procesamiento, el uso de algoritmos eficientes y el desarrollo de políticas de software responsables con el almacenamiento, se ha concebido un proyecto que no solo cumple con sus objetivos técnicos de detección de ataques, sino que también contribuye activamente a un ecosistema digital más eficiente, equitativo y ecológicamente sostenible.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf.

Bibliografía

- [1] Seguridad Social: Cotización / Recaudación de Trabajadores. <https://www.seg-social.es/wps/portal/wss/internet/Trabajadores/CotizacionRecaudacionTrabajadores/36537>. [Último acceso 3 de junio de 2026].